```
    unsigned long const block_size=length/num_threads;

    std::vector<std::future<T> > futures(num_threads-1);     ❸
    std::vector<std::thread> threads(num_threads-1);

    Iterator block_start=first;
    for(unsigned long i=0;i<(num_threads-1);++i)
    {
        Iterator block_end=block_start;
        std::advance(block_end,block_size);
        std::packaged_task<T(Iterator,Iterator)> task(      ❹
            accumulate_block<Iterator,T>());
        futures[i]=task.get_future();                       ❺
        threads[i]=std::thread(std::move(task),block_start,block_end); ❻
        block_start=block_end;
    }
    T last_result=accumulate_block()(block_start,last);     ❼

    std::for_each(threads.begin(),threads.end(),
        std::mem_fn(&std::thread::join));

    T result=init;                                          ❽
    for(unsigned long i=0;i<(num_threads-1);++i)
    {
        result+=futures[i].get();     ❾
    }
    result += last_result;     ❿
    return result;
}
```

The first change is that the function call operator of `accumulate_block` now returns the result directly, rather than taking a reference to somewhere to store it ❶. You're using `std::packaged_task` and `std::future` for the exception safety, so you can use it to transfer the result too. This does require that you explicitly pass in a default-constructed `T` in the call to `std::accumulate` ❷ rather than reusing the supplied `result` value, but that's a minor change.

The next change is that rather than having a vector of results, you have a vector of `futures` ❸ to store a `std::future<T>` for each spawned thread. In the thread-spawning loop, you first create a task for `accumulate_block` ❹. `std::packaged_task<T(Iterator,Iterator)>` declares a task that takes two `Iterator`s and returns a `T`, which is what your function does. You then get the future for that task ❺ and run that task on a new thread, passing in the start and end of the block to process ❻. When the